

SAPIENZA
UNIVERSITÀ DI ROMA

Algoritmi

Esercizi 2 — Linguaggi di programmazione

Esercizi svolti sui capitoli 6–8: programmi in C++, Python e Java, dallo pseudocodice al codice. Ogni esercizio ha la soluzione completa.

Fabio Furini

fabiofurini.github.io

Dipartimento di Ingegneria informatica, automatica e gestionale — Sapienza Università di Roma

Versione online e materiale scaricabile:
fabiofurini.github.io/algoritmi/
github.com/fabiofurini/algoritmi

Anno accademico 2026–2027

Indice

6 Introduzione a C++ **2**

7 Introduzione a Python **26**

8 Introduzione a Java **40**

Capitolo 6

Introduzione a C++

Negli esercizi di questo capitolo i programmi sono scritti, compilati ed eseguiti con le convenzioni del capitolo «Introduzione a `C++`», riassunte qui sotto.

Attenzione

- Il programma di un esercizio è salvato in un file `<name>.cpp`, compilato ed eseguito con i comandi

```
g++ -Wall -std=c++17 -o <name> <name>.cpp
./<name>
```

- Le istanze di test sono array di dimensione fissa dichiarati in `main`, ad esempio `int arr1[] = {9, 12, 3, 16, 19, 11, 12};`, e vengono passate alle funzioni tramite il puntatore al loro primo elemento (`int *a`) insieme alla loro dimensione `n`.
- Gli elementi di un array di dimensione n sono `a[0]`, `a[1]`, …, `a[n-1]`: la componente $a[i]$ di un'istanza scritta come $a = [\dots]$ nel testo corrisponde ad `a[i-1]` nel codice.
- I risultati che una funzione deve restituire oltre al valore di ritorno sono passati per riferimento (`int& x`, `double& x`).
- Quando si stampa un array, ogni valore è seguito da uno spazio, quindi le righe stampate terminano con uno spazio.

Esercizio 6.1 — Introduzione a `C++` ($E1$)

Dato un array a di $n \geq 1$ numeri interi, consideriamo il problema di trovare il valore massimo nell'array.

Domande

1. Scrivere un programma `C++` completo che trova il valore massimo in un array a di n numeri interi. Il programma deve:
   - Definire una funzione `int ArrayMaxValue(int n, int *a)` che implementa l'algoritmo

- Nella funzione `main()`, creare gli array di test e chiamare la funzione
- Stampare i risultati per ogni caso di test

2. Eseguire il programma sulle seguenti istanze:
   - Istanza 1: $a = [9, 12, 3, 16, 19, 11, 12]$ con $n = 7$
   - Istanza 2: $a = [5, 2, 8, 1]$ con $n = 4$
   - Istanza 3: $a = [42]$ con $n = 1$

   Mostrare i comandi di compilazione completi e l'output per tutti i casi di test.

3. Modificare il programma creando una nuova funzione `int ArrayMaxValueWithIndex(int n, int *a, int& idx)` che restituisce il valore massimo e memorizza il suo indice nel parametro per riferimento `idx`. Eseguire questa versione modificata e mostrare l'output.

Soluzione

1. **Complete C++ program:**

```cpp
#include <iostream>
using namespace std;

// Funzione che trova il valore massimo di un array
int ArrayMaxValue(int n, int *a)
{
    int max = a[0];  // Inizializza con il primo elemento

    for (int i = 1; i < n; i++)
    {
        if (a[i] > max)
        {
            max = a[i];  // Aggiorna il massimo
        }
    }

    return max;
}

int main()
{
    // Caso di prova 1
    int arr1[] = {9, 12, 3, 16, 19, 11, 12};
    int n1 = 7;

    cout << "Prova 1 - Array: ";
    for (int i = 0; i < n1; i++)
    {
        cout << arr1[i] << " ";
    }
```

```
    cout << endl;

    int max1 = ArrayMaxValue(n1, arr1);
    cout << "Valore massimo: " << max1 << endl;
    cout << endl;

    // Caso di prova 2
    int arr2[] = {5, 2, 8, 1};
    int n2 = 4;

    cout << "Prova 2 - Array: ";
    for (int i = 0; i < n2; i++)
    {
        cout << arr2[i] << " ";
    }
    cout << endl;

    int max2 = ArrayMaxValue(n2, arr2);
    cout << "Valore massimo: " << max2 << endl;
    cout << endl;

    // Caso di prova 3
    int arr3[] = {42};
    int n3 = 1;

    cout << "Prova 3 - Array: ";
    for (int i = 0; i < n3; i++)
    {
        cout << arr3[i] << " ";
    }
    cout << endl;

    int max3 = ArrayMaxValue(n3, arr3);
    cout << "Valore massimo: " << max3 << endl;

    return 0;
}
```

2. **Compilazione ed esecuzione:**

   Per compilare il programma:

   ```
   g++ -Wall -std=c++17 -o max_value max_value.cpp
   ```

   Per eseguire il programma:

   ```
   ./max_value
   ```

   Output:

```
Prova 1 - Array: 9 12 3 16 19 11 12
Valore massimo: 19

Prova 2 - Array: 5 2 8 1
Valore massimo: 8

Prova 3 - Array: 42
Valore massimo: 42
```

3. **Programma modificato con l'indice:**

```cpp
#include <iostream>
using namespace std;

// Funzione che trova il valore massimo e il suo indice
int ArrayMaxValueWithIndex(int n, int *a, int& idx)
{
    int max = a[0];
    idx = 0;

    for (int i = 1; i < n; i++)
    {
        if (a[i] > max)
        {
            max = a[i];
            idx = i;  // Memorizza l'indice
        }
    }

    return max;
}

int main()
{
    int arr[] = {9, 12, 3, 16, 19, 11, 12};
    int n = 7;
    int index;

    cout << "Array: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;

    int max = ArrayMaxValueWithIndex(n, arr, index);

    cout << "Valore massimo: " << max << endl;
    cout << "Posizione (indice): " << index << endl;

    return 0;
}
```

Output:

```
Array: 9 12 3 16 19 11 12
Valore massimo: 19
Posizione (indice): 4
```

Nota: il valore massimo 19 si trova all'indice 4 (l'elemento di posizione 5, poiché in C++ gli array sono indicizzati a partire da 0). Se il valore massimo compare più di una volta, la funzione memorizza l'indice della sua prima occorrenza, perché `max` e `idx` vengono aggiornati solo quando `a[i] > max` (disuguaglianza stretta).

Esercizio 6.2 — Introduzione a C++ ($E2$)

Dato un array a di $n \geq 1$ numeri interi, consideriamo il problema di trovare il suo valore minimo.

Domande

1. Scrivere un programma C++ completo che trova il valore minimo in un array a di n numeri interi. Il programma deve:
   - Definire una funzione `int ArrayMinValue(int n, int *a)` che implementa l'algoritmo
   - Nella funzione `main()`, creare gli array di test e chiamare la funzione
   - Stampare i risultati per ogni caso di test

2. Eseguire il programma sulle seguenti istanze:
   - Istanza 1: $a = [9, 12, 3, 16, 19, 11, 12]$ con $n = 7$
   - Istanza 2: $a = [15, 7, 22, 4]$ con $n = 4$
   - Istanza 3: $a = [100]$ con $n = 1$

   Mostrare i comandi di compilazione completi e l'output per tutti i casi di test.

3. Modificare il programma creando una nuova funzione `void ArrayMaxMin(int n, int *a, int& max, int& min)` che trova sia il valore massimo sia il valore minimo con un solo passaggio sull'array. Eseguire questa versione modificata e mostrare l'output.

Soluzione

1. **Complete C++ program:**

```cpp
#include <iostream>
using namespace std;

// Funzione che trova il valore minimo di un array
int ArrayMinValue(int n, int *a)
{
    int m = a[0];  // Inizializza con il primo elemento

    for (int i = 1; i < n; i++)
    {
        if (a[i] < m)
        {
            m = a[i];  // Aggiorna il minimo
        }
    }

    return m;
}

int main()
{
    // Caso di prova 1
    int arr1[] = {9, 12, 3, 16, 19, 11, 12};
    int n1 = 7;

    cout << "Prova 1 - Array: ";
    for (int i = 0; i < n1; i++)
    {
        cout << arr1[i] << " ";
    }
    cout << endl;

    int min1 = ArrayMinValue(n1, arr1);
    cout << "Valore minimo: " << min1 << endl;
    cout << endl;

    // Caso di prova 2
    int arr2[] = {15, 7, 22, 4};
    int n2 = 4;

    cout << "Prova 2 - Array: ";
    for (int i = 0; i < n2; i++)
    {
        cout << arr2[i] << " ";
    }
    cout << endl;

    int min2 = ArrayMinValue(n2, arr2);
    cout << "Valore minimo: " << min2 << endl;
    cout << endl;

    // Caso di prova 3
    int arr3[] = {100};
    int n3 = 1;

    cout << "Prova 3 - Array: ";
```

```
    for (int i = 0; i < n3; i++)
    {
        cout << arr3[i] << " ";
    }
    cout << endl;

    int min3 = ArrayMinValue(n3, arr3);
    cout << "Valore minimo: " << min3 << endl;

    return 0;
}
```

2. **Compilazione ed esecuzione:**

   Per compilare il programma:

   ```
   g++ -Wall -std=c++17 -o min_value min_value.cpp
   ```

   Per eseguire il programma:

   ```
   ./min_value
   ```

   Output:

   ```
   Prova 1 - Array: 9 12 3 16 19 11 12
   Valore minimo: 3

   Prova 2 - Array: 15 7 22 4
   Valore minimo: 4

   Prova 3 - Array: 100
   Valore minimo: 100
   ```

3. **Programma modificato per trovare sia il massimo sia il minimo:**

```
#include <iostream>
using namespace std;

// Funzione che trova sia il massimo sia il minimo
void ArrayMaxMin(int n, int *a, int& max, int& min)
{
    max = a[0];
    min = a[0];

    for (int i = 1; i < n; i++)
    {
        if (a[i] > max)
        {
            max = a[i];
```

```
        }
        if (a[i] < min)
        {
            min = a[i];
        }
    }
}

int main()
{
    int arr[] = {9, 12, 3, 16, 19, 11, 12};
    int n = 7;
    int maximum, minimum;

    cout << "Array: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;

    ArrayMaxMin(n, arr, maximum, minimum);

    cout << "Valore massimo: " << maximum << endl;
    cout << "Valore minimo: " << minimum << endl;

    return 0;
}
```

Output:

```
Array: 9 12 3 16 19 11 12
Valore massimo: 19
Valore minimo: 3
```

Nota: questa implementazione trova sia il valore massimo sia il valore minimo con un solo passaggio sull'array. Il numero di confronti è lo stesso che con due funzioni separate, $2(n-1)$, ma l'array viene scorso una volta sola invece di due.

Esercizio 6.3 — Introduzione a C++ ($E3$)

Dato un array a di $n \geq 1$ numeri interi, consideriamo il problema di calcolare la somma e la media di tutti i suoi elementi.

Domande

1. Scrivere un programma C++ completo che calcola la somma e la media degli elementi di un array a di n numeri interi. Il programma deve:

- Definire una funzione `int ArraySum(int n, int *a)` che calcola la somma degli elementi dell'array
- Nella funzione `main()`, creare gli array di test e chiamare la funzione
- Calcolare e stampare sia la somma sia la media per ogni caso di test

2. Eseguire il programma sulle seguenti istanze:
   - Istanza 1: $a = [5, 12, 3, 8]$ con $n = 4$
   - Istanza 2: $a = [10, 20, 30]$ con $n = 3$
   - Istanza 3: $a = [7]$ con $n = 1$

   Mostrare i comandi di compilazione completi e l'output per tutti i casi di test.

3. Modificare il programma creando una nuova funzione `void ArraySumAndAverage(int n, int *a, int& sum, double& avg)` che calcola sia la somma sia la media in un'unica funzione. Eseguire questa versione modificata e mostrare l'output.

Soluzione

1. **Complete C++ program:**

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

// Funzione che calcola la somma degli elementi dell'array
int ArraySum(int n, int *a)
{
    int S = 0;

    for (int i = 0; i < n; i++)
    {
        S = S + a[i];
    }

    return S;
}

int main()
{
    // Caso di prova 1
    int arr1[] = {5, 12, 3, 8};
    int n1 = 4;

    cout << "Prova 1 - Array: ";
    for (int i = 0; i < n1; i++)
    {
        cout << arr1[i] << " ";
    }
```

```
    cout << endl;

    int sum1 = ArraySum(n1, arr1);
    double avg1 = (double)sum1 / n1;
    cout << "Somma: " << sum1 << endl;
    cout << "Media: " << fixed << setprecision(2)
         << avg1 << endl;
    cout << endl;

    // Caso di prova 2
    int arr2[] = {10, 20, 30};
    int n2 = 3;

    cout << "Prova 2 - Array: ";
    for (int i = 0; i < n2; i++)
    {
        cout << arr2[i] << " ";
    }
    cout << endl;

    int sum2 = ArraySum(n2, arr2);
    double avg2 = (double)sum2 / n2;
    cout << "Somma: " << sum2 << endl;
    cout << "Media: " << fixed << setprecision(2)
         << avg2 << endl;
    cout << endl;

    // Caso di prova 3
    int arr3[] = {7};
    int n3 = 1;

    cout << "Prova 3 - Array: ";
    for (int i = 0; i < n3; i++)
    {
        cout << arr3[i] << " ";
    }
    cout << endl;

    int sum3 = ArraySum(n3, arr3);
    double avg3 = (double)sum3 / n3;
    cout << "Somma: " << sum3 << endl;
    cout << "Media: " << fixed << setprecision(2)
         << avg3 << endl;

    return 0;
}
```

2. **Compilazione ed esecuzione:**

   Per compilare il programma:

   ```
   g++ -Wall -std=c++17 -o array_sum array_sum.cpp
   ```

   Per eseguire il programma:

```
./array_sum
```

Output:

```
Prova 1 - Array: 5 12 3 8
Somma: 28
Media: 7.00

Prova 2 - Array: 10 20 30
Somma: 60
Media: 20.00

Prova 3 - Array: 7
Somma: 7
Media: 7.00
```

3. **Programma modificato con la funzione combinata:**

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

// Funzione che calcola sia la somma sia la media
void ArraySumAndAverage(int n, int *a, int& sum, double& avg)
{
    sum = 0;

    for (int i = 0; i < n; i++)
    {
        sum = sum + a[i];
    }

    avg = (double)sum / n;
}

int main()
{
    int arr[] = {5, 12, 3, 8};
    int n = 4;
    int sum;
    double average;

    cout << "Array: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;

    ArraySumAndAverage(n, arr, sum, average);

```

```
34      cout << "Somma: " << sum << endl;
35      cout << "Media: " << fixed << setprecision(2)
36           << average << endl;
37
38      return 0;
39  }
```

Output:

```
Array: 5 12 3 8
Somma: 28
Media: 7.00
```

Nota: il manipolatore `setprecision(2)` serve a formattare la media con 2 cifre decimali. La conversione `(double)` garantisce la divisione in virgola mobile: senza di essa, `sum / n` sarebbe la divisione intera. Poiché la media divide per n, le funzioni richiedono $n \geq 1$.

Esercizio 6.4 — Introduzione a C++ ($E4$)

Dato un array a di n numeri interi e un valore m, consideriamo il problema di determinare se il valore m è presente o meno nell'array.

Domande

1. Scrivere un programma `C++` completo che implementa l'algoritmo di ricerca lineare per un array a di n numeri interi. Il programma deve:
   - Definire una funzione `int LinearSearch(int n, int *a, int m)` che cerca il valore `m` nell'array `a`
   - Restituire l'indice dell'elemento se viene trovato, oppure `-1` se non viene trovato
   - Nella funzione `main()`, creare un array di test e cercare valori diversi
   - Stampare i risultati di ogni ricerca

2. Eseguire il programma con la seguente istanza:
   - Array: $a = [9, 12, 3, 16, 19, 11, 12]$ con $n = 7$
   - Ricerca di: $m = 16$ (elemento presente)
   - Ricerca di: $m = 25$ (elemento non presente)
   - Ricerca di: $m = 9$ (primo elemento)
   - Ricerca di: $m = 12$ (elemento duplicato)

Mostrare i comandi di compilazione completi e l'output per tutti i casi di test.

3. Modificare il programma creando una funzione `int CountOccurrences(int n, int *a, int m)` che conta quante volte il valore `m` compare nell'array. Eseguire questa versione modificata con il valore $m = 12$ nell'array di test e mostrare l'output.

Soluzione

1. **Complete C++ program:**

```cpp
#include <iostream>
using namespace std;

// Funzione che esegue la ricerca lineare
int LinearSearch(int n, int *a, int m)
{
    for (int i = 0; i < n; i++)
    {
        if (a[i] == m)
        {
            return i;  // Trovato: restituisce l'indice
        }
    }

    return -1;  // Non trovato
}

int main()
{
    int arr[] = {9, 12, 3, 16, 19, 11, 12};
    int n = 7;

    cout << "Array: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl << endl;

    // Prova 1: ricerca di 16 (presente)
    int m1 = 16;
    int result1 = LinearSearch(n, arr, m1);
    cout << "Ricerca di " << m1 << ": ";
    if (result1 != -1)
    {
        cout << "Trovato all'indice " << result1 << endl;
    }
    else
    {
        cout << "Non trovato" << endl;
    }

    // Prova 2: ricerca di 25 (assente)
```

```
    int m2 = 25;
    int result2 = LinearSearch(n, arr, m2);
    cout << "Ricerca di " << m2 << ": ";
    if (result2 != -1)
    {
        cout << "Trovato all'indice " << result2 << endl;
    }
    else
    {
        cout << "Non trovato" << endl;
    }

    // Prova 3: ricerca di 9 (primo elemento)
    int m3 = 9;
    int result3 = LinearSearch(n, arr, m3);
    cout << "Ricerca di " << m3 << ": ";
    if (result3 != -1)
    {
        cout << "Trovato all'indice " << result3 << endl;
    }
    else
    {
        cout << "Non trovato" << endl;
    }

    // Prova 4: ricerca di 12 (ripetuto)
    int m4 = 12;
    int result4 = LinearSearch(n, arr, m4);
    cout << "Ricerca di " << m4 << ": ";
    if (result4 != -1)
    {
        cout << "Trovato all'indice " << result4
             << " (prima occorrenza)" << endl;
    }
    else
    {
        cout << "Non trovato" << endl;
    }

    return 0;
}
```

2. **Compilazione ed esecuzione:**

   Per compilare il programma:

   ```
   g++ -Wall -std=c++17 -o linear_search linear_search.cpp
   ```

   Per eseguire il programma:

   ```
   ./linear_search
   ```

   Output:

```
Array: 9 12 3 16 19 11 12

Ricerca di 16: Trovato all'indice 3
Ricerca di 25: Non trovato
Ricerca di 9: Trovato all'indice 0
Ricerca di 12: Trovato all'indice 1 (prima occorrenza)
```

3. **Programma modificato per contare le occorrenze:**

```cpp
#include <iostream>
using namespace std;

// Funzione che conta le occorrenze del valore m
int CountOccurrences(int n, int *a, int m)
{
    int count = 0;

    for (int i = 0; i < n; i++)
    {
        if (a[i] == m)
        {
            count++;
        }
    }

    return count;
}

int main()
{
    int arr[] = {9, 12, 3, 16, 19, 11, 12};
    int n = 7;

    cout << "Array: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;

    int x = 12;
    int count = CountOccurrences(n, arr, x);

    cout << "Il valore " << x << " compare " << count
         << " volte nell'array" << endl;

    return 0;
}
```

Output:

```
Array: 9 12 3 16 19 11 12
Il valore 12 compare 2 volte nell'array
```

Nota: la funzione `LinearSearch` restituisce l'indice della prima occorrenza quando ci sono duplicati. La funzione `CountOccurrences` scorre l'intero array per contare tutte le occorrenze.

Esercizio 6.5 — Introduzione a C++ ($E5$)

Dato un array a di $n \geq 1$ numeri interi, consideriamo il problema di calcolare con un solo passaggio il massimo, il minimo, la somma e la media di tutti gli elementi dell'array.

Domande

1. Scrivere un programma `C++` completo che calcola tutte le statistiche di un array a di n numeri interi con un solo passaggio. Il programma deve:
   - Definire una funzione `void ArrayStatistics(int n, int *a, int& max, int& min, int& sum, double& avg)` che calcola tutte e quattro le statistiche
   - Nella funzione `main()`, creare gli array di test e chiamare la funzione
   - Stampare tutte le statistiche per ogni caso di test

2. Eseguire il programma sulle seguenti istanze:
   - Istanza 1: $a = [9, 12, 3, 16, 19, 11, 12]$ con $n = 7$
   - Istanza 2: $a = [5, 2, 8, 1]$ con $n = 4$

   Mostrare i comandi di compilazione completi e l'output per tutti i casi di test.

3. Discutere i vantaggi di calcolare tutte le statistiche con un solo passaggio rispetto a calcolarle separatamente con più funzioni.

Soluzione

1. **Complete `C++` program:**

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

// Funzione che calcola tutte le statistiche in una sola passata
void ArrayStatistics(int n, int *a, int& max, int& min,
                     int& sum, double& avg)
{
    // Inizializza con il primo elemento
    max = a[0];
    min = a[0];
    sum = a[0];
```

```
13
14      // Elabora gli elementi rimanenti
15      for (int i = 1; i < n; i++)
16      {
17          if (a[i] > max)
18          {
19              max = a[i];
20          }
21          if (a[i] < min)
22          {
23              min = a[i];
24          }
25          sum = sum + a[i];
26      }
27
28      // Calcola la media
29      avg = (double)sum / n;
30  }
31
32  int main()
33  {
34      // Caso di prova 1
35      int arr1[] = {9, 12, 3, 16, 19, 11, 12};
36      int n1 = 7;
37      int max1, min1, sum1;
38      double avg1;
39
40      cout << "Prova 1 - Array: ";
41      for (int i = 0; i < n1; i++)
42      {
43          cout << arr1[i] << " ";
44      }
45      cout << endl;
46
47      ArrayStatistics(n1, arr1, max1, min1, sum1, avg1);
48
49      cout << "Massimo: " << max1 << endl;
50      cout << "Minimo: " << min1 << endl;
51      cout << "Somma: " << sum1 << endl;
52      cout << "Media: " << fixed << setprecision(2)
53           << avg1 << endl;
54      cout << endl;
55
56      // Caso di prova 2
57      int arr2[] = {5, 2, 8, 1};
58      int n2 = 4;
59      int max2, min2, sum2;
60      double avg2;
61
62      cout << "Prova 2 - Array: ";
63      for (int i = 0; i < n2; i++)
64      {
65          cout << arr2[i] << " ";
66      }
67      cout << endl;
68
```

```
    ArrayStatistics(n2, arr2, max2, min2, sum2, avg2);

    cout << "Massimo: " << max2 << endl;
    cout << "Minimo: " << min2 << endl;
    cout << "Somma: " << sum2 << endl;
    cout << "Media: " << fixed << setprecision(2)
         << avg2 << endl;

    return 0;
}
```

2. **Compilazione ed esecuzione:**

   Per compilare il programma:

   ```
   g++ -Wall -std=c++17 -o array_stats array_stats.cpp
   ```

   Per eseguire il programma:

   ```
   ./array_stats
   ```

   Output:

   ```
   Prova 1 - Array: 9 12 3 16 19 11 12
   Massimo: 19
   Minimo: 3
   Somma: 82
   Media: 11.71

   Prova 2 - Array: 5 2 8 1
   Massimo: 8
   Minimo: 1
   Somma: 16
   Media: 4.00
   ```

3. **Vantaggi dell'approccio a un solo passaggio:**

   Vantaggi:

   - **Meno scorrimenti dell'array:** l'array viene scorso una sola volta invece di una volta per ogni statistica
   - **Codice più compatto:** una sola funzione e un solo ciclo invece di più chiamate di funzione
   - **Migliore per dati in flusso:** può elaborare dati che arrivano in sequenza o che non stanno interamente in memoria

- **Minore sovraccarico:** una sola inizializzazione e terminazione di ciclo invece di più cicli

Svantaggi:

- **Meno modulare:** combina più operazioni in una sola funzione, che può essere più difficile da mantenere
- **Meno flessibile:** se serve una sola statistica, la funzione le calcola comunque tutte
- **Riutilizzabilità del codice:** non è facile riutilizzare i singoli calcoli delle statistiche in altri contesti

Raccomandazione: l'approccio a un solo passaggio è preferibile quando:

- servono tutte le statistiche insieme;
- si lavora con array grandi, per i quali è importante ridurre al minimo gli accessi all'array;
- si elaborano dati in flusso o dati che non stanno in memoria.

Le funzioni separate sono migliori quando:

- servono solo statistiche specifiche;
- la chiarezza e la modularità del codice hanno priorità maggiore;
- le singole funzioni possono essere riutilizzate in contesti diversi.

Esercizio 6.6 — Introduzione a C++ ($E6$)

Dato un array bidimensionale a di $m \times n$ numeri interi, consideriamo il problema di calcolare la somma di tutti i suoi elementi.

Domande

1. Scrivere un programma C++ completo che calcola la somma di tutti gli elementi di un array bidimensionale a di $m \times n$ interi. Il programma deve:
   - Definire una funzione `int SumMatrix(int m, int n, int **matrix)` che calcola la somma
   - Nella funzione `main()`, allocare le matrici di test con la memoria dinamica, inizializzarle, chiamare la funzione e stampare i risultati
   - Liberare tutta la memoria allocata dopo l'uso
2. Eseguire il programma sulle seguenti istanze:

- Istanza 1: una matrice 3×4 con valori sequenziali $a[i][j] = (i-1) \cdot n + j$ per $i \in \{1,2,3\}$ e $j \in \{1,2,3,4\}$ (righe e colonne numerate da 1, come nello pseudocodice), cioè

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix}$$

- Istanza 2: una matrice 2×3 con valori $\begin{bmatrix} 3 & 1 & 4 \\ 1 & 5 & 9 \end{bmatrix}$

Mostrare i comandi di compilazione completi e l'output per entrambi i casi di test.

3. Modificare il programma aggiungendo una funzione `void RowSums(int m, int n, int **matrix, int *rowSums)` che calcola la somma di ogni riga e memorizza i risultati nell'array `rowSums`. Eseguire questa versione modificata sull'Istanza 1 e mostrare l'output.

Soluzione

1. **Complete C++ program:**

```cpp
#include <iostream>
using namespace std;

// Funzione che calcola la somma di tutti gli elementi di una matrice
int SumMatrix(int m, int n, int **matrix)
{
    int sum = 0;
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            sum += matrix[i][j];
        }
    }
    return sum;
}

int main()
{
    // Caso di prova 1: matrice 3x4 con valori consecutivi
    int m1 = 3, n1 = 4;

    int** mat1 = new int*[m1];
    for (int i = 0; i < m1; i++)
    {
        mat1[i] = new int[n1];
    }

    int val = 1;
    for (int i = 0; i < m1; i++)
    {
```

```
32          for (int j = 0; j < n1; j++)
33          {
34              mat1[i][j] = val++;
35          }
36      }
37
38      cout << "Prova 1 - Matrice " << m1 << "x" << n1 << ":" << endl;
39      for (int i = 0; i < m1; i++)
40      {
41          for (int j = 0; j < n1; j++)
42          {
43              cout << mat1[i][j] << " ";
44          }
45          cout << endl;
46      }
47      cout << "Somma: " << SumMatrix(m1, n1, mat1) << endl << endl;
48
49      for (int i = 0; i < m1; i++)
50      {
51          delete[] mat1[i];
52      }
53      delete[] mat1;
54
55      // Caso di prova 2: matrice 2x3
56      int m2 = 2, n2 = 3;
57      int data[2][3] = {{3, 1, 4}, {1, 5, 9}};
58
59      int** mat2 = new int*[m2];
60      for (int i = 0; i < m2; i++)
61      {
62          mat2[i] = new int[n2];
63          for (int j = 0; j < n2; j++)
64          {
65              mat2[i][j] = data[i][j];
66          }
67      }
68
69      cout << "Prova 2 - Matrice " << m2 << "x" << n2 << ":" << endl;
70      for (int i = 0; i < m2; i++)
71      {
72          for (int j = 0; j < n2; j++)
73          {
74              cout << mat2[i][j] << " ";
75          }
76          cout << endl;
77      }
78      cout << "Somma: " << SumMatrix(m2, n2, mat2) << endl;
79
80      for (int i = 0; i < m2; i++)
81      {
82          delete[] mat2[i];
83      }
84      delete[] mat2;
85
86      return 0;
87  }
```

2. **Compilazione ed esecuzione:**

   Per compilare il programma:

   ```
   g++ -Wall -std=c++17 -o sum_matrix sum_matrix.cpp
   ```

   Per eseguire il programma:

   ```
   ./sum_matrix
   ```

   Output:

   ```
   Prova 1 - Matrice 3x4:
   1 2 3 4
   5 6 7 8
   9 10 11 12
   Somma: 78

   Prova 2 - Matrice 2x3:
   3 1 4
   1 5 9
   Somma: 23
   ```

3. **Funzione per le somme delle righe:**

```cpp
#include <iostream>
using namespace std;

// Funzione che calcola la somma di tutti gli elementi di una matrice
int SumMatrix(int m, int n, int **matrix)
{
    int sum = 0;
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            sum += matrix[i][j];
        }
    }
    return sum;
}

// Funzione che calcola la somma di ogni riga
void RowSums(int m, int n, int **matrix, int *rowSums)
{
    for (int i = 0; i < m; i++)
    {
```

```
        rowSums[i] = 0;
        for (int j = 0; j < n; j++)
        {
            rowSums[i] += matrix[i][j];
        }
    }
}

int main()
{
    // Caso di prova 1: matrice 3x4 con valori consecutivi
    int m1 = 3, n1 = 4;

    int** mat1 = new int*[m1];
    for (int i = 0; i < m1; i++)
    {
        mat1[i] = new int[n1];
    }

    int val = 1;
    for (int i = 0; i < m1; i++)
    {
        for (int j = 0; j < n1; j++)
        {
            mat1[i][j] = val++;
        }
    }

    cout << "Matrice " << m1 << "x" << n1 << ":" << endl;
    for (int i = 0; i < m1; i++)
    {
        for (int j = 0; j < n1; j++)
        {
            cout << mat1[i][j] << " ";
        }
        cout << endl;
    }
    cout << "Somma totale: " << SumMatrix(m1, n1, mat1) << endl << endl;

    int* rowSums = new int[m1];
    RowSums(m1, n1, mat1, rowSums);

    cout << "Somme delle righe:" << endl;
    for (int i = 0; i < m1; i++)
    {
        cout << "Riga " << (i + 1) << ": " << rowSums[i] << endl;
    }

    delete[] rowSums;
    for (int i = 0; i < m1; i++)
    {
        delete[] mat1[i];
    }
    delete[] mat1;

    return 0;
```

```
79 }
```

Output:

```
Matrice 3x4:
1 2 3 4
5 6 7 8
9 10 11 12
Somma totale: 78

Somme delle righe:
Riga 1: 10
Riga 2: 26
Riga 3: 42
```

Capitolo 7

Introduzione a Python

Negli esercizi di questo capitolo i programmi sono scritti ed eseguiti con le convenzioni del capitolo «Introduzione a `Python`», riassunte qui sotto.

Attenzione

- Il programma di un esercizio è salvato in un file `<name>.py` ed eseguito con il comando

  ```
  python3 <name>.py
  ```

- Le istanze di test sono liste scritte nel codice, ad esempio `arr1 = [9, 12, 3, 16, 19, 11, 12]`, e sono passate alle funzioni senza la loro dimensione: una funzione ottiene la dimensione n di una lista `a` con `len(a)`.
- Gli elementi di una lista di dimensione n sono `a[0]`, `a[1]`, …, `a[n-1]`: la componente $a[i]$ di un'istanza scritta come $a = [\dots]$ nel testo corrisponde ad `a[i-1]` nel codice, e gli indici stampati dai programmi partono da 0.
- Le istruzioni principali sono scritte nel blocco `if __name__ == "__main__":`; una lista si stampa con `print`, che la visualizza fra parentesi quadre.
- I risultati che una funzione deve restituire insieme sono restituiti come una tupla (`return x, y`) e ricevuti con un assegnamento multiplo (`x, y = f(a)`).
- Il formato `:.4f` dentro una f-string visualizza un numero in virgola mobile con esattamente 4 cifre dopo la virgola decimale.

Esercizio 7.1 — Introduzione a `Python` ($E1$)

Dato un array a di $n \geq 1$ numeri interi, consideriamo il problema di trovare il valore massimo nell'array.

Domande

1. Scrivere un programma `Python` completo che trova il valore massimo in una lista a di n numeri interi. Il programma deve:

- definire una funzione `array_max_value(a)` che implementa l'algoritmo;
- nel blocco `__main__`, creare le liste di test e chiamare la funzione;
- stampare i risultati per ogni caso di test.

2. Eseguire il programma sulle seguenti istanze:
   - Istanza 1: $a = [9, 12, 3, 16, 19, 11, 12]$ con $n = 7$
   - Istanza 2: $a = [5, 2, 8, 1]$ con $n = 4$
   - Istanza 3: $a = [42]$ con $n = 1$

   Mostrare il comando che esegue il programma e l'output per tutti i casi di test.

3. Modificare la funzione in modo che restituisca sia il valore massimo sia il suo indice nella lista. Eseguire la versione modificata sull'Istanza 1 e mostrare l'output.

Soluzione

1. **Programma `Python` completo:**

```python
def array_max_value(a):
    """Trova e restituisce il valore massimo della lista a."""
    max_val = a[0]  # Inizializza con il primo elemento

    for i in range(1, len(a)):
        if a[i] > max_val:
            max_val = a[i]  # Aggiorna il massimo

    return max_val


if __name__ == "__main__":
    # Caso di prova 1
    arr1 = [9, 12, 3, 16, 19, 11, 12]
    print(f"Prova 1 - Lista: {arr1}")
    print(f"Valore massimo: {array_max_value(arr1)}")
    print()

    # Caso di prova 2
    arr2 = [5, 2, 8, 1]
    print(f"Prova 2 - Lista: {arr2}")
    print(f"Valore massimo: {array_max_value(arr2)}")
    print()

    # Caso di prova 3
    arr3 = [42]
    print(f"Prova 3 - Lista: {arr3}")
    print(f"Valore massimo: {array_max_value(arr3)}")
```

Per l'Istanza 3 il ciclo `for i in range(1, 1)` non esegue nessuna iterazione, e la funzione restituisce l'unico elemento `a[0]`.

2. **Esecuzione.** Se il programma è salvato nel file `max_value.py`, lo si esegue con il comando

   ```
   python3 max_value.py
   ```

   Output:

   ```
   Prova 1 - Lista: [9, 12, 3, 16, 19, 11, 12]
   Valore massimo: 19

   Prova 2 - Lista: [5, 2, 8, 1]
   Valore massimo: 8

   Prova 3 - Lista: [42]
   Valore massimo: 42
   ```

3. **Funzione modificata che restituisce il valore massimo e il suo indice:**

   ```python
   def array_max_value_with_index(a):
       """Restituisce (valore_massimo, indice) dell'elemento massimo della lista a."""
       max_val = a[0]
       idx = 0

       for i in range(1, len(a)):
           if a[i] > max_val:
               max_val = a[i]
               idx = i  # Memorizza l'indice

       return max_val, idx


   if __name__ == "__main__":
       arr1 = [9, 12, 3, 16, 19, 11, 12]
       max_val, idx = array_max_value_with_index(arr1)
       print(f"Valore massimo: {max_val}, all'indice: {idx}")
   ```

   Output:

   ```
   Valore massimo: 19, all'indice: 4
   ```

   Per l'Istanza 1 la funzione restituisce la tupla $(19,\ 4)$: il valore massimo 19 è all'indice 4 della lista (indici da 0), cioè nella posizione 5 dell'array $a[1..7]$ del testo. Se il valore massimo compare più volte, la funzione restituisce l'indice della sua prima occorrenza, perché `max_val` e `idx` sono aggiornati solo quando `a[i] > max_val` (disuguaglianza stretta).

Esercizio 7.2 — Introduzione a `Python` ($E2$)

Dato un array a di $n \geq 1$ numeri interi, consideriamo il problema di trovare il valore minimo nell'array.

Domande

1. Scrivere un programma `Python` completo che trova il valore minimo in una lista a di n numeri interi. Il programma deve:
   - definire una funzione `array_min_value(a)` che implementa l'algoritmo;
   - nel blocco `__main__`, creare le liste di test e chiamare la funzione;
   - stampare i risultati per ogni caso di test.

2. Eseguire il programma sulle seguenti istanze:
   - Istanza 1: $a = [9, 12, 3, 16, 19, 11, 12]$ con $n = 7$
   - Istanza 2: $a = [5, 2, 8, 1]$ con $n = 4$
   - Istanza 3: $a = [42]$ con $n = 1$

   Mostrare il comando che esegue il programma e l'output per tutti i casi di test.

3. Modificare la funzione in modo che restituisca sia il valore minimo sia il suo indice nella lista. Eseguire la versione modificata sull'Istanza 1 e mostrare l'output.

Soluzione

1. **Programma `Python` completo:**

```python
def array_min_value(a):
    """Trova e restituisce il valore minimo della lista a."""
    min_val = a[0]  # Inizializza con il primo elemento

    for i in range(1, len(a)):
        if a[i] < min_val:
            min_val = a[i]  # Aggiorna il minimo

    return min_val


if __name__ == "__main__":
    # Caso di prova 1
    arr1 = [9, 12, 3, 16, 19, 11, 12]
    print(f"Prova 1 - Lista: {arr1}")
    print(f"Valore minimo: {array_min_value(arr1)}")
```

```
    print()

    # Caso di prova 2
    arr2 = [5, 2, 8, 1]
    print(f"Prova 2 - Lista: {arr2}")
    print(f"Valore minimo: {array_min_value(arr2)}")
    print()

    # Caso di prova 3
    arr3 = [42]
    print(f"Prova 3 - Lista: {arr3}")
    print(f"Valore minimo: {array_min_value(arr3)}")
```

Per l'Istanza 3 il ciclo `for i in range(1, 1)` non esegue nessuna iterazione, e la funzione restituisce l'unico elemento `a[0]`.

2. **Esecuzione.** Se il programma è salvato nel file `min_value.py`, lo si esegue con il comando

   ```
   python3 min_value.py
   ```

   Output:

   ```
   Prova 1 - Lista: [9, 12, 3, 16, 19, 11, 12]
   Valore minimo: 3

   Prova 2 - Lista: [5, 2, 8, 1]
   Valore minimo: 1

   Prova 3 - Lista: [42]
   Valore minimo: 42
   ```

3. **Funzione modificata che restituisce il valore minimo e il suo indice:**

```
def array_min_value_with_index(a):
    """Restituisce (valore_minimo, indice) dell'elemento minimo della
    lista a."""
    min_val = a[0]
    idx = 0

    for i in range(1, len(a)):
        if a[i] < min_val:
            min_val = a[i]
            idx = i  # Memorizza l'indice

    return min_val, idx

```

```python

if __name__ == "__main__":
    arr1 = [9, 12, 3, 16, 19, 11, 12]
    min_val, idx = array_min_value_with_index(arr1)
    print(f"Valore minimo: {min_val}, all'indice: {idx}")
```

Output:

```
Valore minimo: 3, all'indice: 2
```

Per l'Istanza 1 la funzione restituisce la tupla (3, 2): il valore minimo 3 è all'indice 2 della lista (indici da 0), cioè nella posizione 3 dell'array $a[1..7]$ del testo. Se il valore minimo compare più volte, la funzione restituisce l'indice della sua prima occorrenza, perché `min_val` e `idx` sono aggiornati solo quando `a[i] < min_val` (disuguaglianza stretta).

Esercizio 7.3 — Introduzione a Python ($E3$)

Dato un array a di $n \geq 1$ numeri interi, consideriamo il problema di calcolare la somma e la media di tutti i suoi elementi.

Domande

1. Scrivere un programma `Python` completo che calcola la somma e la media degli elementi di una lista a di n numeri interi. Il programma deve:
   - definire una funzione `array_sum(a)` che restituisce la somma di tutti gli elementi;
   - definire una funzione `array_average(a)` che restituisce la media di tutti gli elementi;
   - nel blocco `__main__`, creare le liste di test, chiamare entrambe le funzioni e stampare i risultati, con la media arrotondata a 4 cifre decimali.

2. Eseguire il programma sulle seguenti istanze:
   - Istanza 1: $a = [9, 12, 3, 16, 19, 11, 12]$ con $n = 7$
   - Istanza 2: $a = [5, 2, 8, 1]$ con $n = 4$
   - Istanza 3: $a = [42]$ con $n = 1$

   Mostrare il comando che esegue il programma e l'output per tutti i casi di test.

3. Riunire le due funzioni in un'unica funzione `array_sum_and_average(a)` che restituisce sia la somma sia la media come una tupla. Eseguirla sull'Istanza 1 e mostrare l'output.

Soluzione

1. **Programma `Python` completo:**

```python
def array_sum(a):
    """Calcola e restituisce la somma di tutti gli elementi della lista a.
    """
    total = 0
    for x in a:
        total += x
    return total


def array_average(a):
    """Calcola e restituisce la media di tutti gli elementi della lista a.
    """
    return array_sum(a) / len(a)


if __name__ == "__main__":
    # Caso di prova 1
    arr1 = [9, 12, 3, 16, 19, 11, 12]
    print(f"Prova 1 - Lista: {arr1}")
    print(f"Somma: {array_sum(arr1)}")
    print(f"Media: {array_average(arr1):.4f}")
    print()

    # Caso di prova 2
    arr2 = [5, 2, 8, 1]
    print(f"Prova 2 - Lista: {arr2}")
    print(f"Somma: {array_sum(arr2)}")
    print(f"Media: {array_average(arr2):.4f}")
    print()

    # Caso di prova 3
    arr3 = [42]
    print(f"Prova 3 - Lista: {arr3}")
    print(f"Somma: {array_sum(arr3)}")
    print(f"Media: {array_average(arr3):.4f}")
```

   Il ciclo `for x in a` visita direttamente gli elementi della lista. L'operatore `/` restituisce sempre un `float`, quindi non serve alcuna conversione di tipo (in `C++` la somma deve essere convertita in `double` prima della divisione). La funzione `array_average` richiede $n \geq 1$: per una lista vuota la divisione per `len(a)` $= 0$ ferma il programma con una `ZeroDivisionError`.

2. **Esecuzione.** Se il programma è salvato nel file `array_sum.py`, lo si esegue con il comando

```
python3 array_sum.py
```

Output:

```
Prova 1 - Lista: [9, 12, 3, 16, 19, 11, 12]
Somma: 82
Media: 11.7143

Prova 2 - Lista: [5, 2, 8, 1]
Somma: 16
Media: 4.0000

Prova 3 - Lista: [42]
Somma: 42
Media: 42.0000
```

Per l'Istanza 1 la media esatta è $82/7 = 11.714285\ldots$, visualizzata come 11.7143.

3. **Funzione combinata:**

```python
def array_sum_and_average(a):
    """Restituisce (somma, media) della lista a."""
    total = 0
    for x in a:
        total += x
    avg = total / len(a)
    return total, avg


if __name__ == "__main__":
    arr1 = [9, 12, 3, 16, 19, 11, 12]
    s, avg = array_sum_and_average(arr1)
    print(f"Somma: {s}, Media: {avg:.4f}")
```

Output:

```
Somma: 82, Media: 11.7143
```

La funzione combinata visita la lista una sola volta, mentre `array_average` del primo programma chiama `array_sum` e visita la lista una seconda volta.

Esercizio 7.4 — Introduzione a Python ($E4$)

Dato un array a di n numeri interi e un valore m, consideriamo il problema di determinare se il valore m è presente o no nell'array.

Domande

1. Scrivere un programma `Python` completo che esegue una **ricerca lineare** in una lista a. Il programma deve:
   - definire una funzione `linear_search(a, m)` che cerca il valore `m` nella lista `a`;
   - restituire l'indice della prima occorrenza di `m` se viene trovato, oppure `-1` se non viene trovato;
   - nel blocco `__main__`, eseguire i casi di test e stampare i risultati.

2. Eseguire il programma sulla lista $a = [9, 12, 3, 16, 19, 11, 12]$ con i seguenti valori:
   - $m = 16$ (presente nella lista)
   - $m = 12$ (presente due volte: si restituisce l'indice della prima occorrenza)
   - $m = 7$ (non presente nella lista)

   Mostrare il comando che esegue il programma e l'output per tutti i casi di test.

3. Modificare la funzione in modo che restituisca la **lista di tutti gli indici** in cui compare il valore m (invece della sola prima occorrenza). Eseguirla con $m = 12$ e mostrare l'output.

Soluzione

1. **Programma `Python` completo:**

```python
def linear_search(a, m):
    """Cerca il valore m nella lista a.
    Restituisce l'indice della prima occorrenza, oppure -1 se non è
    presente."""
    for i in range(len(a)):
        if a[i] == m:
            return i  # Trovato: restituisce l'indice
    return -1         # Non trovato


def print_result(a, m):
    """Cerca m in a e stampa il risultato."""
    idx = linear_search(a, m)
    if idx != -1:
        print(f"Valore {m} trovato all'indice {idx}")
    else:
        print(f"Valore {m} non trovato")


if __name__ == "__main__":
    a = [9, 12, 3, 16, 19, 11, 12]
    print(f"Lista: {a}")
    print()
```

```
23
24      print_result(a, 16)
25      print_result(a, 12)
26      print_result(a, 7)
```

L'istruzione `return i` termina la funzione, e quindi il ciclo, alla prima occorrenza di `m`; l'istruzione `return -1` viene raggiunta solo quando il ciclo ha visitato l'intera lista senza trovare `m`. Si usa il valore `-1` perché non può essere un indice della lista. La funzione ausiliaria `print_result` evita di scrivere tre volte le stesse istruzioni.

2. **Esecuzione.** Se il programma è salvato nel file `linear_search.py`, lo si esegue con il comando

   ```
   python3 linear_search.py
   ```

   Output:

   ```
   Lista: [9, 12, 3, 16, 19, 11, 12]

   Valore 16 trovato all'indice 3
   Valore 12 trovato all'indice 1
   Valore 7 non trovato
   ```

   Il valore 12 si trova negli indici 1 e 6, e la funzione restituisce il primo. Nell'array $a[1..7]$ del testo, gli indici 3 e 1 corrispondono alle posizioni 4 e 2.

3. **Funzione che restituisce tutte le occorrenze:**

```python
def linear_search_all(a, m):
    """Restituisce la lista di tutti gli indici in cui il valore m compare
     in a.
    Restituisce una lista vuota se m non è presente."""
    indices = []
    for i in range(len(a)):
        if a[i] == m:
            indices.append(i)
    return indices


if __name__ == "__main__":
    a = [9, 12, 3, 16, 19, 11, 12]

    m = 12
    result = linear_search_all(a, m)
    if result:  # Una lista non vuota è vera, una lista vuota è falsa
        print(f"Valore {m} trovato agli indici: {result}")
    else:
        print(f"Valore {m} non trovato")
```

Output:

```
Valore 12 trovato agli indici: [1, 6]
```

A differenza di `linear_search`, questa funzione visita sempre l'intera lista, perché non può fermarsi alla prima occorrenza. La lista `indices` parte vuota e cresce con `append`; quando è usata come condizione, una lista vuota vale `False`.

Esercizio 7.5 — Introduzione a Python ($E5$)

Dato un array a di $n \geq 1$ numeri interi, consideriamo il problema di calcolare in un solo passaggio il massimo, il minimo, la somma e la media di tutti gli elementi dell'array.

Domande

1. Scrivere un programma Python completo che calcola in un **solo passaggio** le quattro statistiche (massimo, minimo, somma, media) di una lista a di n numeri interi. Il programma deve:
   - definire una funzione `array_stats(a)` che calcola i quattro valori con un solo ciclo;
   - restituire i quattro valori come una tupla `(max_val, min_val, total, avg)`;
   - nel blocco `__main__`, eseguire i casi di test e stampare i risultati.

2. Eseguire il programma sulle seguenti istanze:
   - Istanza 1: $a = [9, 12, 3, 16, 19, 11, 12]$ con $n = 7$
   - Istanza 2: $a = [5, 2, 8, 1]$ con $n = 4$
   - Istanza 3: $a = [42]$ con $n = 1$

   Mostrare il comando che esegue il programma e l'output per tutti i casi di test.

3. Modificare la funzione in modo che restituisca anche gli indici dell'elemento massimo e dell'elemento minimo. Eseguirla sull'Istanza 1 e mostrare l'output.

Soluzione

1. **Programma Python completo:**

```python
def array_stats(a):
    """Calcola massimo, minimo, somma e media in un solo passaggio.
    Restituisce (max_val, min_val, total, avg)."""
    max_val = a[0]
```

```
    min_val = a[0]
    total = a[0]

    for i in range(1, len(a)):
        if a[i] > max_val:
            max_val = a[i]
        if a[i] < min_val:
            min_val = a[i]
        total += a[i]

    avg = total / len(a)
    return max_val, min_val, total, avg


def print_stats(k, a):
    """Stampa le statistiche del caso di prova k."""
    max_v, min_v, s, avg = array_stats(a)
    print(f"Prova {k} - Lista: {a}")
    print(f"  Massimo: {max_v}")
    print(f"  Minimo:  {min_v}")
    print(f"  Somma:   {s}")
    print(f"  Media:   {avg:.4f}")


if __name__ == "__main__":
    print_stats(1, [9, 12, 3, 16, 19, 11, 12])
    print()
    print_stats(2, [5, 2, 8, 1])
    print()
    print_stats(3, [42])
```

Le tre variabili sono inizializzate con il primo elemento, e il ciclo visita una sola volta gli altri $n - 1$ elementi: a ogni iterazione l'elemento corrente è confrontato con il massimo e con il minimo, e viene sommato alla somma.

2. **Esecuzione.** Se il programma è salvato nel file `array_stats.py`, lo si esegue con il comando

   ```
   python3 array_stats.py
   ```

   Output:

   ```
   Prova 1 - Lista: [9, 12, 3, 16, 19, 11, 12]
     Massimo: 19
     Minimo:  3
     Somma:   82
     Media:   11.7143

   Prova 2 - Lista: [5, 2, 8, 1]
   ```

```
  Massimo: 8
  Minimo:  1
  Somma:   16
  Media:   4.0000

Prova 3 - Lista: [42]
  Massimo: 42
  Minimo:  42
  Somma:   42
  Media:   42.0000
```

3. **Funzione estesa che restituisce anche gli indici:**

```python
def array_stats_with_indices(a):
    """Calcola tutte le statistiche, comprese le posizioni.
    Restituisce (max_val, max_idx, min_val, min_idx, total, avg)."""
    max_val = a[0]
    max_idx = 0
    min_val = a[0]
    min_idx = 0
    total = a[0]

    for i in range(1, len(a)):
        if a[i] > max_val:
            max_val = a[i]
            max_idx = i
        if a[i] < min_val:
            min_val = a[i]
            min_idx = i
        total += a[i]

    avg = total / len(a)
    return max_val, max_idx, min_val, min_idx, total, avg


if __name__ == "__main__":
    a = [9, 12, 3, 16, 19, 11, 12]
    max_v, max_i, min_v, min_i, s, avg = array_stats_with_indices(a)
    print(f"Lista:   {a}")
    print(f"Max:     {max_v} all'indice {max_i}")
    print(f"Min:     {min_v} all'indice {min_i}")
    print(f"Somma:   {s}")
    print(f"Media:   {avg:.4f}")
```

Output:

```
Lista:   [9, 12, 3, 16, 19, 11, 12]
Max:     19 all'indice 4
Min:     3 all'indice 2
Somma:   82
Media:   11.7143
```

Il massimo 19 è all'indice 4 e il minimo 3 all'indice 2 (indici da 0), cioè nelle posizioni 5 e 3 dell'array $a[1..7]$ del testo.

Capitolo 8

Introduzione a Java

Negli esercizi di questo capitolo i programmi sono scritti, compilati ed eseguiti con le convenzioni del capitolo «Introduzione a `Java`», riassunte qui sotto.

Attenzione

- Il programma di un esercizio è una classe `Main`, salvata nel file `Main.java` (ogni esercizio in una cartella propria), compilata ed eseguita con i comandi

```
javac Main.java
java Main
```

- I metodi sono metodi `static` della classe `Main`. Le istanze di test sono array scritti nel codice, ad esempio `int[] arr1 = {9, 12, 3, 16, 19, 11, 12};`, e sono passate ai metodi senza la loro dimensione: un metodo ottiene la dimensione n di un array `a` con `a.length`.
- Gli elementi di un array di dimensione n sono `a[0]`, `a[1]`, ..., `a[n-1]`: la componente $a[i]$ di un'istanza scritta come $a = [\dots]$ nel testo corrisponde ad `a[i-1]` nel codice, e gli indici stampati dai programmi partono da 0.
- Un metodo restituisce al più un valore: i risultati che un metodo deve restituire insieme sono restituiti in un array (`int[]` o `double[]`).
- Quando un array è stampato con un ciclo, ogni valore è seguito da uno spazio, quindi le righe stampate terminano con uno spazio.
- Il formato `%.4f` di `printf` visualizza un numero in virgola mobile con esattamente 4 cifre dopo la virgola decimale. Gli output sono quelli di un calcolatore con impostazioni in inglese: con le impostazioni in italiano `printf` scrive la virgola decimale (`11,7143`).

Esercizio 8.1 — Introduzione a `Java` ($E1$)

Dato un array a di $n \geq 1$ numeri interi, consideriamo il problema di trovare il valore massimo nell'array.

Domande

1. Scrivere un programma `Java` completo che trova il valore massimo in un array a di n numeri interi. Il programma deve:
   - definire un metodo statico `int arrayMaxValue(int[] a)` dentro la classe `Main`;
   - nel metodo `main`, creare gli array di test e chiamare il metodo;
   - stampare i risultati per ogni caso di test.

2. Eseguire il programma sulle seguenti istanze:
   - Istanza 1: $a = [9, 12, 3, 16, 19, 11, 12]$ con $n = 7$
   - Istanza 2: $a = [5, 2, 8, 1]$ con $n = 4$
   - Istanza 3: $a = [42]$ con $n = 1$

   Mostrare i comandi di compilazione ed esecuzione e l'output per tutti i casi di test.

3. Modificare il metodo in modo che restituisca sia il valore massimo sia il suo indice. Poiché un metodo `Java` restituisce un solo valore, restituire un array `int[]` di due elementi: `{maxValue, maxIndex}`. Eseguire la versione modificata sull'Istanza 1 e mostrare l'output.

Soluzione

1. **Programma `Java` completo:**

```java
public class Main {

    static int arrayMaxValue(int[] a) {
        int max = a[0]; // Inizializza con il primo elemento

        for (int i = 1; i < a.length; i++) {
            if (a[i] > max) {
                max = a[i]; // Aggiorna il massimo
            }
        }
        return max;
    }

    public static void main(String[] args) {
        int[] arr1 = {9, 12, 3, 16, 19, 11, 12};
        System.out.print("Prova 1 - Array: ");
        for (int x : arr1) System.out.print(x + " ");
        System.out.println();
        System.out.println("Valore massimo: " + arrayMaxValue(arr1));
        System.out.println();

        int[] arr2 = {5, 2, 8, 1};
        System.out.print("Prova 2 - Array: ");
```

```
        for (int x : arr2) System.out.print(x + " ");
        System.out.println();
        System.out.println("Valore massimo: " + arrayMaxValue(arr2));
        System.out.println();

        int[] arr3 = {42};
        System.out.print("Prova 3 - Array: ");
        for (int x : arr3) System.out.print(x + " ");
        System.out.println();
        System.out.println("Valore massimo: " + arrayMaxValue(arr3));
    }
}
```

Gli array sono stampati con il ciclo `for` esteso, che visita direttamente i loro elementi. Per l'Istanza 3 la condizione `1 < a.length` è falsa fin dall'inizio, il ciclo non esegue nessuna iterazione, e il metodo restituisce l'unico elemento `a[0]`.

2. **Compilazione ed esecuzione:**

```
javac Main.java
java Main
```

Output:

```
Prova 1 - Array: 9 12 3 16 19 11 12
Valore massimo: 19

Prova 2 - Array: 5 2 8 1
Valore massimo: 8

Prova 3 - Array: 42
Valore massimo: 42
```

3. **Metodo modificato che restituisce il valore massimo e il suo indice:**

```
public class Main {

    // Restituisce int[]{maxValue, maxIndex}
    static int[] arrayMaxValueWithIndex(int[] a) {
        int max = a[0];
        int idx = 0;

        for (int i = 1; i < a.length; i++) {
            if (a[i] > max) {
                max = a[i];
                idx = i; // Memorizza l'indice
            }
        }
        return new int[]{max, idx};
```

```
    }

    public static void main(String[] args) {
        int[] arr1 = {9, 12, 3, 16, 19, 11, 12};
        int[] result = arrayMaxValueWithIndex(arr1);
        System.out.println("Valore massimo: " + result[0]
                           + ", all'indice: " + result[1]);
    }
}
```

Output:

```
Valore massimo: 19, all'indice: 4
```

Per l'Istanza 1 il metodo restituisce l'array $\{19,\ 4\}$: il valore massimo 19 è all'indice 4 (indici da 0), cioè nella posizione 5 dell'array $a[1..7]$ del testo. Se il valore massimo compare più volte, il metodo restituisce l'indice della sua prima occorrenza, perché `max` e `idx` sono aggiornati solo quando `a[i] > max` (disuguaglianza stretta).

Esercizio 8.2 — Introduzione a `Java` ($E2$)

Dato un array a di $n \geq 1$ numeri interi, consideriamo il problema di trovare il valore minimo nell'array.

Domande

1. Scrivere un programma `Java` completo che trova il valore minimo in un array a di n numeri interi. Il programma deve:
   - definire un metodo statico `int arrayMinValue(int[] a)` dentro la classe `Main`;
   - nel metodo `main`, creare gli array di test e chiamare il metodo;
   - stampare i risultati per ogni caso di test.

2. Eseguire il programma sulle seguenti istanze:
   - Istanza 1: $a = [9, 12, 3, 16, 19, 11, 12]$ con $n = 7$
   - Istanza 2: $a = [5, 2, 8, 1]$ con $n = 4$
   - Istanza 3: $a = [42]$ con $n = 1$

   Mostrare i comandi di compilazione ed esecuzione e l'output per tutti i casi di test.

3. Modificare il metodo in modo che restituisca sia il valore minimo sia il suo indice. Poiché un metodo `Java` restituisce un solo valore, restituire un array `int[]` di due elementi: `{minValue, minIndex}`. Eseguire la versione modificata sull'Istanza 1 e mostrare l'output.

Soluzione

1. **Programma Java completo:**

```java
public class Main {

    static int arrayMinValue(int[] a) {
        int min = a[0]; // Inizializza con il primo elemento

        for (int i = 1; i < a.length; i++) {
            if (a[i] < min) {
                min = a[i]; // Aggiorna il minimo
            }
        }
        return min;
    }

    public static void main(String[] args) {
        int[] arr1 = {9, 12, 3, 16, 19, 11, 12};
        System.out.print("Prova 1 - Array: ");
        for (int x : arr1) System.out.print(x + " ");
        System.out.println();
        System.out.println("Valore minimo: " + arrayMinValue(arr1));
        System.out.println();

        int[] arr2 = {5, 2, 8, 1};
        System.out.print("Prova 2 - Array: ");
        for (int x : arr2) System.out.print(x + " ");
        System.out.println();
        System.out.println("Valore minimo: " + arrayMinValue(arr2));
        System.out.println();

        int[] arr3 = {42};
        System.out.print("Prova 3 - Array: ");
        for (int x : arr3) System.out.print(x + " ");
        System.out.println();
        System.out.println("Valore minimo: " + arrayMinValue(arr3));
    }
}
```

Gli array sono stampati con il ciclo `for` esteso, che visita direttamente i loro elementi. Per l'Istanza 3 la condizione `1 < a.length` è falsa fin dall'inizio, il ciclo non esegue nessuna iterazione, e il metodo restituisce l'unico elemento `a[0]`.

2. **Compilazione ed esecuzione:**

```
javac Main.java
java Main
```

Output:

```
Prova 1 - Array: 9 12 3 16 19 11 12
```

```
Valore minimo: 3

Prova 2 - Array: 5 2 8 1
Valore minimo: 1

Prova 3 - Array: 42
Valore minimo: 42
```

3. **Metodo modificato che restituisce il valore minimo e il suo indice:**

```java
public class Main {

    // Restituisce int[]{minValue, minIndex}
    static int[] arrayMinValueWithIndex(int[] a) {
        int min = a[0];
        int idx = 0;

        for (int i = 1; i < a.length; i++) {
            if (a[i] < min) {
                min = a[i];
                idx = i; // Memorizza l'indice
            }
        }
        return new int[]{min, idx};
    }

    public static void main(String[] args) {
        int[] arr1 = {9, 12, 3, 16, 19, 11, 12};
        int[] result = arrayMinValueWithIndex(arr1);
        System.out.println("Valore minimo: " + result[0]
                           + ", all'indice: " + result[1]);
    }
}
```

Output:

```
Valore minimo: 3, all'indice: 2
```

Per l'Istanza 1 il metodo restituisce l'array $\{3,\ 2\}$: il valore minimo 3 è all'indice 2 (indici da 0), cioè nella posizione 3 dell'array $a[1..7]$ del testo. Se il valore minimo compare più volte, il metodo restituisce l'indice della sua prima occorrenza, perché `min` e `idx` sono aggiornati solo quando `a[i] < min` (disuguaglianza stretta).

Esercizio 8.3 — Introduzione a Java ($E3$)

Dato un array a di $n \geq 1$ numeri interi, consideriamo il problema di calcolare la somma e la media di tutti i suoi elementi.

Domande

1. Scrivere un programma `Java` completo che calcola la somma e la media degli elementi di un array a di n numeri interi. Il programma deve:
   - definire un metodo statico `int arraySum(int[] a)` che restituisce la somma;
   - definire un metodo statico `double arrayAverage(int[] a)` che restituisce la media;
   - nel metodo `main`, creare gli array di test, chiamare entrambi i metodi e stampare i risultati, con la media arrotondata a 4 cifre decimali.

2. Eseguire il programma sulle seguenti istanze:
   - Istanza 1: $a = [9, 12, 3, 16, 19, 11, 12]$ con $n = 7$
   - Istanza 2: $a = [5, 2, 8, 1]$ con $n = 4$
   - Istanza 3: $a = [42]$ con $n = 1$

   Mostrare i comandi di compilazione ed esecuzione e l'output per tutti i casi di test.

3. Scrivere un unico metodo `arraySumAndAverage(int[] a)` che restituisce sia la somma sia la media. Poiché un metodo `Java` restituisce un solo valore, restituire il risultato come un array `double[]` di due elementi: `{sum, average}`. Eseguirlo sull'Istanza 1 e mostrare l'output.

Soluzione

1. **Programma `Java` completo:**

```java
public class Main {

    static int arraySum(int[] a) {
        int total = 0;
        for (int i = 0; i < a.length; i++) {
            total += a[i];
        }
        return total;
    }

    static double arrayAverage(int[] a) {
        return (double) arraySum(a) / a.length;
    }

    static void printTest(int k, int[] a) {
        System.out.print("Prova " + k + " - Array: ");
        for (int x : a) System.out.print(x + " ");
        System.out.println();
        System.out.println("Somma: " + arraySum(a));
        System.out.printf("Media: %.4f%n", arrayAverage(a));
    }
```

```
22
23      public static void main(String[] args) {
24          int[] arr1 = {9, 12, 3, 16, 19, 11, 12};
25          printTest(1, arr1);
26          System.out.println();
27
28          int[] arr2 = {5, 2, 8, 1};
29          printTest(2, arr2);
30          System.out.println();
31
32          int[] arr3 = {42};
33          printTest(3, arr3);
34      }
35  }
```

La conversione `(double)` è necessaria: senza di essa, `arraySum(a) / a.length` sarebbe la divisione intera, e per l'Istanza 1 la media sarebbe $82/7 = 11$. Il metodo `printTest` stampa l'array e i due risultati, ed evita di scrivere tre volte le stesse istruzioni.

2. **Compilazione ed esecuzione:**

```
javac Main.java
java Main
```

Output:

```
Prova 1 - Array: 9 12 3 16 19 11 12
Somma: 82
Media: 11.7143

Prova 2 - Array: 5 2 8 1
Somma: 16
Media: 4.0000

Prova 3 - Array: 42
Somma: 42
Media: 42.0000
```

Per l'Istanza 1 la media esatta è $82/7 = 11.714285\ldots$, visualizzata come 11.7143.

3. **Metodo combinato:**

```
1  public class Main {
2
3      // Restituisce double[]{somma, media}
4      static double[] arraySumAndAverage(int[] a) {
5          int total = 0;
6          for (int x : a) total += x;
```

```
        double avg = (double) total / a.length;
        return new double[]{total, avg};
    }

    public static void main(String[] args) {
        int[] arr1 = {9, 12, 3, 16, 19, 11, 12};
        double[] result = arraySumAndAverage(arr1);
        System.out.printf("Somma: %.0f, Media: %.4f%n",
                          result[0], result[1]);
    }
}
```

Output:

```
Somma: 82, Media: 11.7143
```

L'intero `total` è convertito automaticamente in un `double` quando viene memorizzato nell'array `double[]`; per questo motivo è stampato con il formato `%.0f` (nessuna cifra decimale) invece di `%d`, che accetta solo interi.

Esercizio 8.4 — Introduzione a Java ($E4$)

Dato un array a di n numeri interi e un valore m, consideriamo il problema di determinare se il valore m è presente o no nell'array.

Domande

1. Scrivere un programma `Java` completo che esegue una **ricerca lineare** in un array a. Il programma deve:
   - definire un metodo statico `int linearSearch(int[] a, int m)` che cerca il valore `m` nell'array `a`;
   - restituire l'indice della prima occorrenza di `m` se viene trovato, oppure `-1` se non viene trovato;
   - nel metodo `main`, eseguire i casi di test e stampare i risultati.

2. Eseguire il programma sull'array $a = [9, 12, 3, 16, 19, 11, 12]$ con i seguenti valori:
   - $m = 16$ (presente nell'array)
   - $m = 12$ (presente due volte: si restituisce l'indice della prima occorrenza)
   - $m = 7$ (non presente nell'array)

   Mostrare i comandi di compilazione ed esecuzione e l'output per tutti i casi di test.

3. Modificare il metodo in modo che restituisca un array `int[]` contenente **tutti gli indici** in cui compare il valore m (invece della sola prima occorrenza). Usare una `ArrayList<Integer>` dentro il metodo e convertirla in un `int[]` prima di restituirla. Eseguirlo con $m = 12$ e mostrare l'output.

Soluzione

1. **Programma Java completo:**

```java
public class Main {

    static int linearSearch(int[] a, int m) {
        for (int i = 0; i < a.length; i++) {
            if (a[i] == m) {
                return i; // Trovato: restituisce l'indice
            }
        }
        return -1; // Non trovato
    }

    static void printResult(int[] a, int m) {
        int idx = linearSearch(a, m);
        if (idx != -1) System.out.println("Valore " + m + " trovato all'indice " + idx);
        else           System.out.println("Valore " + m + " non trovato");
    }

    public static void main(String[] args) {
        int[] a = {9, 12, 3, 16, 19, 11, 12};
        System.out.print("Array: ");
        for (int x : a) System.out.print(x + " ");
        System.out.println();
        System.out.println();

        printResult(a, 16);
        printResult(a, 12);
        printResult(a, 7);
    }
}
```

L'istruzione `return i` termina il metodo, e quindi il ciclo, alla prima occorrenza di `m`; l'istruzione `return -1` viene raggiunta solo quando il ciclo ha visitato l'intero array senza trovare `m`. Si usa il valore `-1` perché non può essere un indice dell'array.

2. **Compilazione ed esecuzione:**

```
javac Main.java
java Main
```

Output:

```
Array: 9 12 3 16 19 11 12

Valore 16 trovato all'indice 3
Valore 12 trovato all'indice 1
Valore 7 non trovato
```

Il valore 12 si trova negli indici 1 e 6, e il metodo restituisce il primo. Nell'array $a[1..7]$ del testo, gli indici 3 e 1 corrispondono alle posizioni 4 e 2.

3. **Metodo che restituisce tutte le occorrenze:**

```java
import java.util.ArrayList;

public class Main {

    static int[] linearSearchAll(int[] a, int m) {
        ArrayList<Integer> indices = new ArrayList<>();
        for (int i = 0; i < a.length; i++) {
            if (a[i] == m) {
                indices.add(i);
            }
        }
        // Conversione della ArrayList<Integer> in un int[]
        int[] result = new int[indices.size()];
        for (int i = 0; i < indices.size(); i++) {
            result[i] = indices.get(i);
        }
        return result;
    }

    public static void main(String[] args) {
        int[] a = {9, 12, 3, 16, 19, 11, 12};
        int m = 12;
        int[] occurrences = linearSearchAll(a, m);

        if (occurrences.length > 0) {
            System.out.print("Valore " + m + " trovato agli indici: ");
            for (int idx : occurrences) System.out.print(idx + " ");
            System.out.println();
        } else {
            System.out.println("Valore " + m + " non trovato");
        }
    }
}
```

Output:

```
Valore 12 trovato agli indici: 1 6
```

Il numero di occorrenze non è noto in anticipo, quindi gli indici sono prima raccolti in una `ArrayList`, che cresce con `add`; poi si alloca e si riempie un array della dimensione giusta `indices.size()`. A differenza di `linearSearch`, questo metodo visita sempre l'intero array. Se `m` non è presente, il metodo restituisce un array di lunghezza 0.

Esercizio 8.5 — Introduzione a Java ($E5$)

Dato un array a di $n \geq 1$ numeri interi, consideriamo il problema di calcolare in un solo passaggio il massimo, il minimo, la somma e la media di tutti gli elementi dell'array.

Domande

1. Scrivere un programma `Java` completo che calcola in un **solo passaggio** le quattro statistiche (massimo, minimo, somma, media) di un array a di n numeri interi. Il programma deve:
   - definire un metodo statico `arrayStats(int[] a)` che calcola i quattro valori con un solo ciclo;
   - restituire i risultati come un array `double[]` di quattro elementi: `{max, min, sum, average}`;
   - nel metodo `main`, eseguire i casi di test e stampare i risultati.

2. Eseguire il programma sulle seguenti istanze:
   - Istanza 1: $a = [9, 12, 3, 16, 19, 11, 12]$ con $n = 7$
   - Istanza 2: $a = [5, 2, 8, 1]$ con $n = 4$
   - Istanza 3: $a = [42]$ con $n = 1$

   Mostrare i comandi di compilazione ed esecuzione e l'output per tutti i casi di test.

3. Modificare il metodo in modo che restituisca anche gli indici dell'elemento massimo e dell'elemento minimo, come un array `double[]` di sei elementi: `{max, maxIdx, min, minIdx, sum, average}`. Eseguirlo sull'Istanza 1 e mostrare l'output.

Soluzione

1. **Programma `Java` completo:**

```java
public class Main {

    // Restituisce double[]{massimo, minimo, somma, media}
    static double[] arrayStats(int[] a) {
        int max = a[0];
        int min = a[0];
        int total = a[0];

        for (int i = 1; i < a.length; i++) {
            if (a[i] > max) max = a[i];
            if (a[i] < min) min = a[i];
            total += a[i];
```

```
        }

        double avg = (double) total / a.length;
        return new double[]{max, min, total, avg};
    }

    static void printStats(int k, int[] a) {
        double[] s = arrayStats(a);
        System.out.print("Prova " + k + " - Array: ");
        for (int x : a) System.out.print(x + " ");
        System.out.println();
        System.out.println("  Massimo: " + (int) s[0]);
        System.out.println("  Minimo:  " + (int) s[1]);
        System.out.println("  Somma:   " + (int) s[2]);
        System.out.printf("  Media:   %.4f%n", s[3]);
    }

    public static void main(String[] args) {
        printStats(1, new int[]{9, 12, 3, 16, 19, 11, 12});
        System.out.println();
        printStats(2, new int[]{5, 2, 8, 1});
        System.out.println();
        printStats(3, new int[]{42});
    }
}
```

Le tre variabili sono inizializzate con il primo elemento, e il ciclo visita una sola volta gli altri $n - 1$ elementi. I quattro risultati hanno tipi di dato diversi, quindi sono tutti memorizzati come `double` nell'array restituito; i risultati interi sono riconvertiti con `(int)` prima della stampa, in modo che siano visualizzati senza la parte decimale (`19` invece di `19.0`). L'espressione `new int[]{...}` crea un array direttamente nella chiamata.

2. **Compilazione ed esecuzione:**

```
javac Main.java
java Main
```

Output:

```
Prova 1 - Array: 9 12 3 16 19 11 12
  Massimo: 19
  Minimo:  3
  Somma:   82
  Media:   11.7143

Prova 2 - Array: 5 2 8 1
  Massimo: 8
  Minimo:  1
  Somma:   16
  Media:   4.0000
```

```
Prova 3 - Array: 42
  Massimo: 42
  Minimo:  42
  Somma:   42
  Media:   42.0000
```

3. **Metodo esteso che restituisce anche gli indici:**

```java
public class Main {

    // Restituisce double[]{max, maxIdx, min, minIdx, somma, media}
    static double[] arrayStatsWithIndices(int[] a) {
        int max = a[0], maxIdx = 0;
        int min = a[0], minIdx = 0;
        int total = a[0];

        for (int i = 1; i < a.length; i++) {
            if (a[i] > max) { max = a[i]; maxIdx = i; }
            if (a[i] < min) { min = a[i]; minIdx = i; }
            total += a[i];
        }

        double avg = (double) total / a.length;
        return new double[]{max, maxIdx, min, minIdx, total, avg};
    }

    public static void main(String[] args) {
        int[] a = {9, 12, 3, 16, 19, 11, 12};
        double[] r = arrayStatsWithIndices(a);
        System.out.println("Max:   " + (int) r[0] + " all'indice " + (int) r[1]);
        System.out.println("Min:   " + (int) r[2] + " all'indice " + (int) r[3]);
        System.out.println("Somma: " + (int) r[4]);
        System.out.printf("Media: %.4f%n", r[5]);
    }
}
```

Output:

```
Max:   19 all'indice 4
Min:   3 all'indice 2
Somma: 82
Media: 11.7143
```

Il massimo 19 è all'indice 4 e il minimo 3 all'indice 2 (indici da 0), cioè nelle posizioni 5 e 3 dell'array $a[1..7]$ del testo.